

Programming In C-HDOC DAY_19

Program

The algorithm for calculating the Semester Grade Point Average (SGPA) and Cumulative Grade Point Average

(CGPA) for 'm' semesters is given below. Each semester has 'n' courses, where 'n' may vary between semesters.

Assume that the student has passed all courses. Prepare a test-case table covering valid, invalid, and boundary

inputs; write the C program based on the given algorithm; compile and execute it.

ALGORITHM

1. Start.
2. Define the variables and their types: global_id, m, n, semester, course, code, credit, GP, semester_TC, semester_TGC, cumulative_TC, and cumulative_TGC as integers; grade as a character; and SGPA and CGPA as floating-point variables.
3. Get global_id.
4. Validate global_id: If global_id is not positive, display an appropriate error message and request it again. Repeat until a positive value is entered.
5. Get m.
6. Validate m: If m is not positive, display an appropriate error message and request it again. Repeat until a positive value is entered.
7. Set the cumulative values: cumulative_TC = 0, cumulative_TGC = 0
8. Repeat Steps 8.1 to 8.8 for m semesters:

HDOC DAY 19 REPORT

8.1. Set the semester totals:

$\text{semester_TC} = 0, \text{semester_TGC} = 0$

8.2. Get n.

8.3. Validate n: If n is not positive, display an appropriate error message and request it again. Repeat until a positive

value is entered.

8.4. Repeat Steps 8.4.1 to 8.4.9 for n courses:

8.4.1. Get code.

8.4.2. Validate code: If it is not positive, display an error message and request it again. Repeat until a positive

value is entered.

8.4.3. Get credit.

8.4.4. Validate credit: If it is not positive, display an error message and request it again. Repeat until a positive

value is entered.

8.4.5. Get grade.

8.4.6. Validate grade: Accept only O, A, B, C, D, E, or P. If the grade is invalid, display an error message and

request it again. Repeat until a valid grade is entered.

8.4.7. Find GP corresponding to grade using the university's conversion scale:

$O = 10, A = 9, B = 8, C = 7, D = 6, E = 5, P = 4$

8.4.8. Calculate the semester total grade credits:

$\text{semester_TGC} = \text{semester_TGC} + (\text{GP} * \text{credit})$

8.4.9. Calculate the semester total credits:

$\text{semester_TC} = \text{semester_TC} + \text{credit}$

8.5. Calculate the SGPA of the current semester:

$\text{SGPA} = \text{semester_TGC} / \text{semester_TC}$

8.6. Display semester and SGPA.

8.7. Add the semester grade credits to the cumulative grade credits:

HDOC DAY 19 REPORT

$\text{cumulative_TGC} = \text{cumulative_TGC} + \text{semester_TGC}$

8.8. Add the semester credits to the cumulative credits:

$\text{cumulative_TC} = \text{cumulative_TC} + \text{semester_TC}$

9. Calculate the CGPA:

$\text{CGPA} = \text{cumulative_TGC} / \text{cumulative_TC}$

10. Display `global_id`, `m`, `cumulative_TC`, and `CGPA`.

11. Stop

C PROGRAM CODE

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int global_id, m, n, semester, course;
```

```
    int code, credit, GP;
```

```
    int semester_TC, semester_TGC;
```

```
    int cumulative_TC, cumulative_TGC;
```

```
    char grade;
```

```
    float SGPA, CGPA;
```

```
    /* Get and validate global_id */
```

```
    do
```

```
    {
```

```
        printf("Enter Global ID: ");
```

```
        scanf("%d", &global_id);
```

```
        if (global_id <= 0)
```

```
            printf("Error: Global ID must be positive.\n");
```

HDOC DAY 19 REPORT

```
} while (global_id <= 0);

/* Get and validate number of semesters */
do
{
    printf("Enter number of semesters: ");
    scanf("%d", &m);

    if (m <= 0)
        printf("Error: Number of semesters must be positive.\n");

} while (m <= 0);

/* Initialize cumulative values */
cumulative_TC = 0;
cumulative_TGC = 0;

/* Process each semester */
for (semester = 1; semester <= m; semester++)
{
    semester_TC = 0;
    semester_TGC = 0;

    /* Get and validate number of courses */
    do
    {
        printf("\nEnter number of courses for Semester %d: ", semester);
        scanf("%d", &n);
```

HDOC DAY 19 REPORT

```
    if (n <= 0)

        printf("Error: Number of courses must be positive.\n");

} while (n <= 0);

/* Process each course */
for (course = 1; course <= n; course++)
{
    /* Get and validate course code */
    do
    {
        printf("Enter code for Course %d: ", course);
        scanf("%d", &code);

        if (code <= 0)
            printf("Error: Course code must be positive.\n");

    } while (code <= 0);

    /* Get and validate credit */
    do
    {
        printf("Enter credit for Course %d: ", course);
        scanf("%d", &credit);

        if (credit <= 0)
            printf("Error: Credit must be positive.\n");
```

HDOC DAY 19 REPORT

```
} while (credit <= 0);

/* Get and validate grade */
do
{
    printf("Enter grade for Course %d (O/A/B/C/D/E/P): ", course);
    scanf(" %c", &grade);

    if (grade != 'O' && grade != 'A' &&
        grade != 'B' && grade != 'C' &&
        grade != 'D' && grade != 'E' &&
        grade != 'P')
    {
        printf("Error: Invalid grade. Enter O, A, B, C, D, E, or P.\n");
    }

} while (grade != 'O' && grade != 'A' &&
        grade != 'B' && grade != 'C' &&
        grade != 'D' && grade != 'E' &&
        grade != 'P');

/* Convert grade to grade point */
switch (grade)
{
    case 'O':
        GP = 10;
        break;
```

HDOC DAY 19 REPORT

```
    case 'A':
        GP = 9;
        break;
    case 'B':
        GP = 8;
        break;
    case 'C':
        GP = 7;
        break;
    case 'D':
        GP = 6;
        break;
    case 'E':
        GP = 5;
        break;
    case 'P':
        GP = 4;
        break;
}

/* Calculate semester totals */
semester_TGC = semester_TGC + (GP * credit);
semester_TC = semester_TC + credit;
}

/* Calculate SGPA */
SGPA = (float)semester_TGC / semester_TC;
```

HDOC DAY 19 REPORT

```
printf("Semester %d SGPA = %.2f\n", semester, SGPA);

/* Update cumulative totals */

cumulative_TGC = cumulative_TGC + semester_TGC;
cumulative_TC = cumulative_TC + semester_TC;
}

/* Calculate CGPA */

CGPA = (float)cumulative_TGC / cumulative_TC;

/* Display final result */

printf("\n----- Final Result ----- \n");
printf("Global ID      : %d\n", global_id);
printf("Number of Semesters: %d\n", m);
printf("Cumulative Credits : %d\n", cumulative_TC);
printf("CGPA              : %.2f\n", CGPA);

return 0;
}
```